

How MafutaPlan Achieved Multiple Linear Regression

Component-based fuel price prediction for Nairobi, Kenya

Project	MafutaPlan
Author	Ryan Alfred Nyambati
Registration	SCT222-0195/2021
Institution	Jomo Kenyatta University of Agriculture and Technology
Model	Pooled multiple linear regression
Test cycle	April 2026

Purpose: explain, in reproducible terms, how the application converted verified EPRA component records into a one-cycle-ahead fuel-price prediction and how the result was evaluated.

1. What the model was designed to do

MafutaPlan estimates the maximum Nairobi retail price for the next pricing cycle. The prediction is based on component information from the immediately preceding cycle. The target is continuous (KSh per litre), so linear regression is an appropriate transparent baseline.

Prediction unit

One row represents one fuel product in one input cycle, paired with that fuel's published retail price in the following target cycle. Super Petrol, Diesel, and Kerosene are pooled into one model.

Model inputs

Feature	Meaning in the application
Landed_Cost	Imported product, freight, financing, port, and exchange-rate effects
Distribution_Storage	Handling, storage, pipeline, losses, depot, and delivery costs
Margins	Approved wholesale and retail margins
Stabilization_Adjustment	Signed subsidy, deficit, surplus, or reconciliation adjustment
Taxes_Levies	Excise, VAT, road, petroleum, regulatory, and related charges
Fuel_Diesel	1 for Diesel; 0 otherwise
Fuel_Kerosene	1 for Kerosene; 0 otherwise

Super Petrol is the reference fuel because both fuel indicator variables equal zero for its rows.

Target

Target_Retail_Price is the official Nairobi maximum retail price in the following cycle. The feature rows and targets are stored in **data/component_prediction_dataset.csv**.

2. The regression formulation

The application uses scikit-learn's ordinary least-squares **LinearRegression** estimator. It learns an intercept and one coefficient for each numerical or encoded input.

Predicted price = $b_0 + b_1(\text{Landed Cost}) + b_2(\text{Distribution and Storage}) + b_3(\text{Margins}) + b_4(\text{Stabilization}) + b_5(\text{Taxes and Levies}) + b_6(\text{Diesel}) + b_7(\text{Kerosene})$

The estimator chooses the coefficients that minimise the sum of squared differences between observed training prices and fitted training prices. In code, **design_matrix()** creates the seven columns, and **fit_linear_regression()** calls `model.fit(X, y)`.

Chronological split and leakage prevention

Random splitting would allow later regulatory cycles to help predict earlier ones. MafutaPlan instead sorts rows by Target_Cycle, reserves the latest complete cycle as the test set, and trains only on earlier targets. This preserves the direction of time.

Stage	Cycles / records	Role
Training	Sep 2024 to Mar 2026; 30 rows	Learn coefficients
Testing	Apr 2026; 3 rows	Unseen chronological holdout

Implementation sequence

Step	Operation
1	Load and validate the model-ready component dataset.
2	Sort by target cycle and fuel product.
3	Reserve the latest complete target cycle.
4	Encode Diesel and Kerosene indicators.
5	Fit LinearRegression on the earlier records.
6	Predict the three held-out April prices.
7	Calculate absolute error, percentage error, MAE, and RMSE.

3. Held-out April 2026 results

The model achieved a mean absolute error (MAE) of **15.41 KSh/L** and a root mean squared error (RMSE) of **19.30 KSh/L** across the three held-out fuel products.

Fuel	Official	Predicted	Abs. error	% error
Diesel	196.63	167.86	28.77	14.63%
Kerosene	152.78	153.23	0.45	0.29%
Super Petrol	197.60	180.59	17.01	8.61%

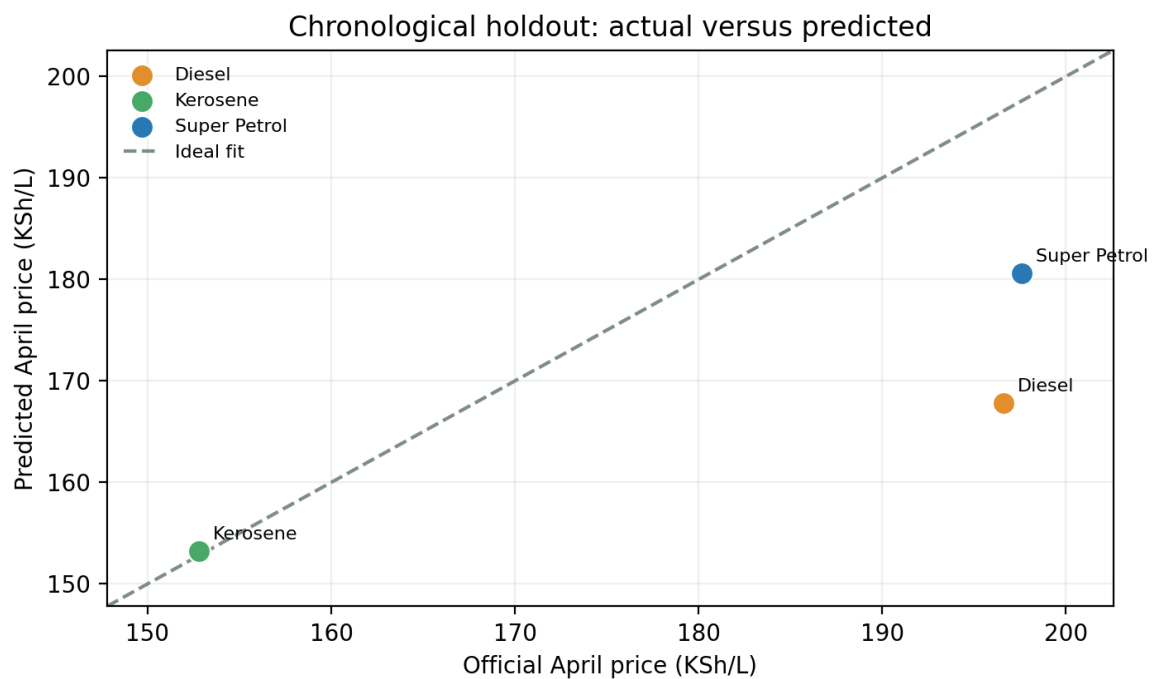


Figure 1. The dashed diagonal is perfect agreement. Kerosene is close to the line, while Diesel and Super Petrol are under-predicted.

MAE is the average absolute miss in KSh/L. RMSE gives larger errors more weight, which is why the Diesel error has a stronger influence.

4. What the fitted coefficients mean

A positive coefficient means the fitted prediction rises when that input rises and the other inputs are held constant. A negative coefficient means the fitted prediction falls. These are model associations, not proof that an input causes the price change.



Figure 2. Feature coefficients only; the intercept is omitted from the graph so that the feature bars remain readable.

Term	Coefficient
Intercept	134.108520
Landed_Cost	0.526436
Distribution_Storage	-61.973967
Margins	8.945869
Stabilization_Adjustment	0.300605
Taxes_Levies	1.737948
Fuel_Diesel	-9.775039
Fuel_Kerosene	-2.348368

Caution: the component variables use different ranges and are often correlated. Raw coefficient magnitudes therefore should not be used as a simple ranking of real-world importance.

5. How the app presents the model

The Fuel Price Prediction page selects a fuel, displays its predicted April price, official April price, absolute error, and March component inputs. It then renders two linear-regression graphs:

Graph	Purpose
Actual versus predicted	Shows the three held-out observations against an ideal-fit line.
Learned coefficients	Shows the direction and size of the fitted feature associations.

Reproducibility map

File	Responsibility
src/data.py	Loads and validates the model-ready data.
src/modeling.py	Builds X, fits LinearRegression, predicts, and evaluates.
app.py	Displays metrics, tables, and interactive regression graphs.
data/component_prediction_dataset.csv	Stores one-cycle-ahead training and test rows.
DATA_PROVENANCE.md	Documents source and verification rules.

Limitations

The component panel is small and discontinuous. Regulatory changes, tax revisions, and stabilization decisions can shift abruptly. The model is pooled across three fuels and is intended as an academic, explainable baseline; it does not replace EPRA and should not be treated as a station-level price guarantee.

Conclusion

MafutaPlan achieved multiple linear regression by converting verified fuel build-up components into seven model features, pairing each input cycle with the following retail-price cycle, fitting ordinary least squares only on earlier observations, and testing on an unseen April 2026 cycle. The implementation is transparent, reproducible, and now visually explained in both the application and this document.